LIrAs User Study

Thank you for considering participation in this study conducted by researchers at Politecnico di Milano.

This study investigates how users understand and use LIrAs, a textual domain-specific language for specifying coordinated interaction patterns involving multiple agents.

You will receive a short introduction to LIrAs, complete a small set of tasks involving the interpretation, creation, and modification of LIrAs specifications, and answer a brief questionnaire about your experience.

The study is expected to take approximately **[60 minutes]**.

Please complete the study individually using a desktop or laptop computer. During the tasks, use only the material provided as part of the study and do not use generative AI tools or assistance from another person.

* Indica una domanda obbligatoria

Study information and informed consent

PURPOSE OF THE STUDY

The purpose of this study is to evaluate the effort required to understand and use LIrAs and to identify aspects of the language that may need improvement. The results may be included in scientific publications, presentations, and reports concerning the evaluation of LIrAs.

DATA & PRIVACY

We will collect:

- your answers to the study tasks;
- the LIrAs specifications that you submit;
- your questionnaire ratings and comments;
- broad categories describing your technical background and prior experience.

We will not ask for your name, email address, exact age, employer, institution, or other direct identifiers. We will not intentionally collect sensitive personal information.

Please do not include your name, contact details, employer, institution, or information identifying another person in any open-text response.

USE AND STORAGE OF DATA

The collected data will be used only for research purposes related to the evaluation of LIrAs. Access to the raw responses will be limited to the research team.

Results will be reported primarily in aggregate form. Short excerpts from open-text responses may be quoted in scientific publications only after removing information that could identify the participant.

Raw participant responses will not be made publicly available. Study materials, aggregate results, and analysis procedures may be included in a public research artifact.

CONTACT INFORMATION

For questions about the study, please contact:

Alberto Tagliaferro

Politecnico di Milano

alberto.tagliaferro@polimi.it

Research team: Alberto Tagliaferro, Livia Lestingi, and Matteo G. Rossi

1. **Adult participation requirement** *

   Participation in this study is limited to adults. If you cannot confirm this statement, please close this page without submitting the form.

   Contrassegna solo un ovale.

   ◯ I confirm that I am 18 years of age or older

Participant background and prior experience

The following questions help us characterize the study sample and interpret the study results.

They concern only broad categories of technical background and prior experience.

Please answer based on your experience **before starting this study**.

2. Which option best describes your current primary role? *

Contrassegna solo un ovale.

- ◯ Bachelor's or Master's student
- ◯ PhD student
- ◯ Academic researcher
- ◯ Industry practitioner
- ◯ Other role

3. Which area best describes your main educational or professional background? *

Contrassegna solo un ovale.

- ◯ Computer Science or Software Engineering
- ◯ Robotics, Automation, Control, or Human–Robot Interaction
- ◯ Other technical/scientific field
- ◯ Other field

4. How many years of programming experience did you have before this study? *

Contrassegna solo un ovale.

- ◯ No programming experience
- ◯ Less than 1 year
- ◯ 1–3 years
- ◯ More than 3

5. How would you rate your previous experience in each of the following areas? *

None: I had never studied or used it.
Basic: I knew what it was but had no practical experience.
Limited: I had used it in one course, tutorial, or small project.
Moderate: I had used it in several courses or projects.
Extensive: I used it regularly in professional or research activities.

Contrassegna solo un ovale per riga.

	None	Basic	Limited	Moderate	Extensive
Textual domain-specific or specification languages, such as PlantUML or workflow DSLs	◯	◯	◯	◯	◯
Software or system modeling notations, such as UML, state machines, or Petri nets	◯	◯	◯	◯	◯
Uppaal or automata-based modeling and verification	◯	◯	◯	◯	◯
Formal methods / Formal verification (other than Uppaal)	◯	◯	◯	◯	◯
Robotics, multi-agent systems, or human–robot interaction	◯	◯	◯	◯	◯

6. Before being invited to this study, how familiar were you with LIrAs? *

Contrassegna solo un ovale.

- ◯ I had never heard of LIrAs.
- ◯ I had heard of LIrAs but had not read documentation or used it.
- ◯ I had read or seen material about LIrAs but had not used it.
- ◯ I had previously used LIrAs.
- ◯ I am not sure.

7. How comfortable are you reading technical instructions and documentation in English? *

Contrassegna solo un ovale.

- ◯ Beginner: Can understand and use basic, familiar phrases
- ◯ Intermediate: Can understand the main points of standard input on familiar matters
- ◯ Advanced: Can use language flexibly and effectively for complex purposes
- ◯ Proficient: Can understand virtually everything heard or read with ease
- ◯ Native speaker

LIrAs tutorial

This short tutorial introduces the LIrAs concepts needed for this study. It is intended as training, not as an assessment.

You may return to this section during the later tasks. Please use only the material provided in this study and do not consult external sources, other people, or generative AI tools.

No previous knowledge is required.

Estimated reading time: approximately **8 minutes**.

What LIrAs describes

LIrAs is a textual domain-specific language for describing reusable interaction patterns involving multiple agents.

A LIrAs pattern specifies:

- the participating agents, locations and resources;
- one sequence of actions for each agent;
- when different agents synchronize and when they may proceed independently;
- conditions that alter action execution;
- optional success and failure conditions.

Atomic actions and predicates are domain-specific and are provided externally. For each study task, you will receive the complete list of actions and predicates that may be used.

Actions describe something that an agent can execute.

Some actions do not require parameters:

- *"takeOff"* instructs the agent executing the sequence to take off.

Other actions require parameters to specify a location, resource, or other target:

- "*moveTo loadingArea*" instructs the agent to move to the location loadingArea;
- "*pick item1*" instructs the agent to pick the resource item1.

Predicates describe conditions that evaluate to either true or false.

Some predicates do not require parameters because they represent a single, unambiguous condition:

- "*emergencyStopActive*" is true when the system-wide emergency stop is active.

Other predicates require parameters:

- "*position(Robot, loadingArea)*" is true when Robot is at loadingArea;
- "*holding(Robot, item1)*" is true when Robot is holding item1.

Predicates can be combined with Boolean operators:

- And: "&&"
- Or: " || "
- Not: " ! "

1. Basic pattern structure

A LIrAs specification begins with the name of the pattern. The participating agents (grouped by type), locations, and resources are then declared. Finally, each agent has its own action sequence.

In an agent declaration such as "*ground_robots: Robot"*, "*Robot"* is the identifier used for the agent sequence and in conditions referring to that agent, of the type "*ground_robot*".

Example:
****

Pattern_Name

agents:
  ground_robots: Robot1, Robot2
  aerial_robots: Drone
locations: loadingArea, shelf
resources: item1

Robot1:
  1: moveTo loadingArea

Robot2:
  1: moveTo shelf

Drone:
  1: takeOff

****

Actions may use the declared agents, locations, and resources as parameters. For example, "*moveTo loadingArea"* applies the "*moveTo*" action to the declared location "*loadingArea*".

2. Sub-sequences and synchronization

A number followed by a colon starts a **sub-sequence**:

e.g. "2: pick item1"

Any following action without another numeric label belongs to the same sub-sequence.

Sub-sequences with the same number in different agent sequences start at the same time. The following sub-sequence can begin only after the agents have completed their current sub-sequences.

When one agent finishes earlier than another, it waits at the synchronization point.

If no action is specified for an agent in a sub-sequence, that sub-sequence is considered immediately completed for that agent. The agent therefore waits, if necessary, until the next synchronization point.

Example:
****

Robot:
1: moveTo loadingArea
2: pick item1
3: moveTo shelf

Drone:
1: takeOff
2: moveTo shelf
4: inspect item1
land

****

The two actions in sub-sequence 2 start together. If the "*Robot*" finishes "*pick item1*" before the "*Drone*" finishes "*moveTo shelf*", the "*Robot*" waits.

Sub-sequence 3 starts only after both agents have completed sub-sequence 2. Here the "*Robot*" moves to the shelf. Once the "*Robot*" reaches its destination, the "*Drone*" starts sub-sequence 4, first inspecting the item carried by the "*Robot*" and then landing.

3. Continuations

A continuation allows one agent to proceed independently across multiple sub-sequences.

A range such as: " *2-4: inspect shelf "* means that the agent synchronizes with the other agents at the beginning of sub-sequence 2 and again after the end of sub-sequence 4. It does not synchronize at the intermediate boundary between sub-sequences 2 and 4.
Example:
****

Robot:
2: pick item1
3: moveTo shelf
4: place item1

Drone:
2-3: inspect shelf
4: hover Robot

****

Both agents start sub-sequence 2 together. Once the "*Robot"* completes "*pick item1*", it may immediately start "*moveTo shelf*", even if the Drone is still executing inspect shelf.

Before starting sub-sequence 4, both agents must have completed their respective executions over sub-sequences 2–3.

4. Conditions and action control

LIrAs conditions may change the execution of individual actions or entire sub-sequences.

4.1 If/else

****

moveTo shelf ***if*** *pathClear* ***else*** *adjustPosition*

****

The main "*moveTo*" action is executed while "*pathClear" predicate* is true. If it becomes false, the main action is interrupted and "*adjustPosition"* is executed instead. If "*pathClear*" becomes true again, execution returns to the main action.

4.2 Until

****

inspect shelf ***until*** *freeSlotFound*

****

The "*inspect shelf*" action may complete normally, but it terminates early if "*freeSlotFound*" becomes true.

4.3 Skip

****

skip if *obstacleDetected*

****

If "*obstacleDetected*" is true, the current sub-sequence of this agent terminates immediately. The agent then proceeds toward the next synchronization point, waiting for the other agents.

4.4 Restart

****

restart if *graspFailed*

****

If "*graspFailed*" is true, the current sub-sequence of this agent is executed again.

Note: Scope of skip and restart. "*skip*" and "*restart*" affect only the current sub-sequence of the agent in whose sequence they appear. They do not directly restart or skip the actions of the other agents.

5. Pattern termination

A pattern normally terminates when all agents have completed their sequences.

LlrAs can also specify conditions that terminate the pattern early with respect to the normal termination when the sequence is completed:

****

success *delivered(item1)*
failure *SoC(Robot) < 0.15 || SoC(Drone) < 0.15*

****

When a "*success*" condition becomes true, the entire pattern terminates erlier with a successful outcome, even if there are some ongoing actions.

When a "*failure*" condition becomes true, the entire pattern terminates with a failure outcome.

A pattern may define a "*success*" condition, a "*failure*" condition, both, or neither.

6. Complete example

Scenario Description:
Consider a maintenance task involving a service robot and a human technician. While the "*Technician*" prepares a panel, the "*Robot*" retrieves a wrench and brings it to the maintenance area. They then coordinate the tool handover.

****
Maintenance_Assistance

agents:
  robots: Robot
  humans: Technician
locations: toolRack, maintenanceArea
resources: wrench

Robot:
  1: moveTo toolRack
  2: pick wrench
  3: moveTo Technician if routeClear(Technician) else wait
  4: handOver wrench

Technician:
  1-3: prepare panel
  4: receive wrench

failure emergencyStopActive || unsafeProximity(Robot, Technician)

****

The "*Robot*" performs sub-sequences 1 to 3 while the "*Technician*" prepares the panel as a continuation over the same interval. Therefore, they start together but do not synchronize at the intermediate boundaries. If the route to the maintenance area is not clear, the "*Robot*" waits and resumes moving when the route becomes clear.
The pattern fails if the emergency stop is activated or if the "*Robot*" and the "*Technician*" enter an unsafe proximity.

A possible equivalent code for the sequences can be:
****
Robot:
  1: moveTo toolRack
    pick wrench
    moveTo Technician if routeClear(Technician) else wait
  2: handOver wrench
Technician:
  1: prepare panel
  2: receive wrench
****
The agents synchronize at sub-sequence 2, where "*handOver wrench*" and "*receive wrench*" start simultaneously.

Training checks

The following questions check the concepts introduced in the tutorial. They are part of the training and are not included in the main task-performance measures.

Please answer based on your current understanding. The correct explanations are provided on the next page. Please do not return and change your answers after viewing the explanations, as they will not be evaluated.

8. **Consider the following specification fragment:** *

****

Robot:
1: moveTo loadingArea
2: pick item1
3: moveTo shelf

Drone:
1: takeOff
2: inspect item1
3: land

****

The "*Robot*" completes "*pick item1*" before the "*Drone*" completes "*inspect item1*". What happens next?

Contrassegna solo un ovale.

- The "Robot" immediately starts "moveTo shelf".
- The "Robot" waits until the "Drone" completes "inspect item1"; then both agents start sub-sequence 3.
- The Drone's "inspect item1" action is interrupted.
- The "Robot" skips sub-sequence 3.

9. **Consider the following specification fragment:** *

****

Robot:

2: pick item1
3: moveTo shelf
4: place item1

Drone:
2-3: inspect shelf
4: hover Robot

****

The "*Robot*" completes "*pick item1*" while the "*Drone*" is still executing "*inspect shelf*". What may the "*Robot*" do next?

Contrassegna solo un ovale.

- It must wait for the "Drone" before starting "moveTo shelf".
- It may start "moveTo shelf"; both agents must synchronize again before sub-sequence 4.
- It skips "moveTo shelf" and immediately starts sub-sequence 4.
- The pattern terminates because the agents completed their actions at different times.

10. **Consider the following instruction:** *

moveTo shelf if pathClear else adjustPosition

During execution, "*pathClear*" becomes false. What happens?

Contrassegna solo un ovale.

- "moveTo shelf" continues because the condition is checked only once, when the action starts.
- "moveTo shelf" is interrupted, and "adjustPosition" is executed; if "pathClear" becomes true again, "moveTo shelf" resumes.
- The current sub-sequence is skipped.
- The entire pattern terminates with a failure outcome.

7. Training check explanations

Check 1 — Synchronization

Correct answer:

- The "Robot" waits until the "Drone" completes "inspect item1"; then both agents start sub-sequence 3.

Sub-sequences with the same label start together. Sub-sequence 3 can begin only after both the "*Robot*" and the "*Drone*" have completed sub-sequence 2. The "*Robot*" therefore waits for the "*Drone*".

Check 2 — Continuation

Correct answer:

- It may start "moveTo shelf"; both agents must synchronize again before sub-sequence 4.

The label 2-3 defines a continuation. The "*Drone*" does not synchronize with the "*Robot*" at the intermediate boundary between sub-sequences 2 and 3. The "*Robot*" may therefore start "*moveTo shelf*" immediately after completing "*pick item1*". Both agents synchronize again before sub-sequence 4.

Check 3 — Conditional action

Correct answer:

- "moveTo shelf" is interrupted, and "adjustPosition" is executed; if "pathClear" becomes true again, "moveTo shelf" resumes.

In an "***if*** ... ***else*** ..." instruction, the condition may alter execution while the instruction is active. When "*pathClear*" becomes false, "*moveTo shelf*" is interrupted and "*adjustPosition*" is executed. The main action resumes if the condition becomes true again.

Please return to the tutorial if any of these rules are unclear.

T1 — Understanding a LIrAs pattern

In this task, you will read a LIrAs pattern and answer questions about its behavior.

You may consult the tutorial. Please do not use external sources, generative AI tools, or assistance from another person.

Please spend no more than approximately **5 minutes** on this task. If you are unsure, provide your best answer and continue.

Greenhouse Inspection Scenario

Consider a greenhouse inspection involving an autonomous rover and a human agronomist. "r*ow1*" is the specific crop row selected for inspection.

The "*Rover*" performs a preliminary scan of the row, while the "*Agronomist*" collects a sensor kit and moves to it. They then collaborate on a close inspection: the Rover provides continuous lighting while the Agronomist examines the crop. After the inspection, both return to the supply area, where the Agronomist stores the sensor kit.

The following actions and predicates are available. Underlined terms indicate the kind of value expected by an action or predicate; they are placeholders and should not be written literally.

In this scenario:

- agent type may be "*inspection_rovers"* or "*human_agronomists"*.
- location may be "*supplyArea"* or "*row1"*;
- cropRow refers to "*row1"*;
- resource refers to "*sensorKit"*.

Actions:

- *moveTo* location: moves the executing agent to the specified location;
- *scan* cropRow: performs a preliminary scan of the specified crop row;
- *provideLighting* cropRow: continuously illuminates the specified crop row; in this task, the action remains active until the condition attached through until becomes true;
- *collect* resource: collects the specified resource;
- *inspect* cropRow: performs a close inspection of the specified crop row, when the action completes, *inspectionCompleted*(cropRow) becomes true;
- *wait*: waits without performing another activity;
- *store* resource: stores the specified resource.

Predicates:

- *diseaseDetected*(cropRow) is true when the Rover's preliminary scan detects signs of disease in the specified crop row;
- *rowIlluminated*(cropRow) is true while the Rover is providing lighting to the specified crop row;
- *inspectionCompleted*(cropRow) becomes true when the Agronomist completes the close inspection of the specified crop row;
- *emergencyStopActive* is true when the system-wide emergency stop is active.

****
Collaborative_Crop_Inspection

agents:
inspection_rovers: Rover
human_agronomists: Agronomist
locations: supplyArea, row1
resources: sensorKit

Rover:
1: moveTo row1
2: scan row1 until diseaseDetected(row1)
3: provideLighting row1 until inspectionCompleted(row1)
4: moveTo supplyArea

Agronomist:
1-2: collect sensorKit
moveTo row1
3: inspect row1 if rowIlluminated(row1) else wait
4: moveTo supplyArea
store sensorKit

failure emergencyStopActive

****

11. The "*Rover*" completes "*moveTo row1*" while the "*Agronomist*" is still executing the continuation over sub-sequences 1–2. What may the "*Rover*" do next? *

Contrassegna solo un ovale.

- The "Rover" must wait until the "Agronomist" has completed both "collect sensorKit" and "moveTo Row1".
- The "Rover" may start "scan row1" without waiting for the "Agronomist" at the boundary between sub-sequences 1 and 2.
- The Agronomist's current action is interrupted so that both agents can start sub-sequence 2 together.
- The "Rover" skips sub-sequence 2 and proceeds directly to sub-sequence 3.

12. During sub-sequence 2, diseaseDetected(row1) becomes true while the Agronomist is still moving toward the crop row. What happens? *

Contrassegna solo un ovale.

- The entire pattern terminates with a failure outcome.
- The Rover's scanning action terminates early, but the "Rover" waits at the end of sub-sequence 2 until the "Agronomist" completes the continuation.
- The "Rover" restarts the scanning action.
- The "Agronomist" skips "moveTo row1" and immediately starts the inspection.

13. Which statement best describes sub-sequence 3? *

Contrassegna solo un ovale.

- The "Rover" completes the lighting first; the "Agronomist" then begins inspecting row1.
- The "Rover" lights row1 until the inspection ends; the "Agronomist" inspects only while the row is illuminated.
- The "Agronomist" inspects continuously; the "Rover" remains idle throughout sub-sequence 3.
- Each agent may enter sub-sequence 4 immediately after completing its own current action.

14. **Consider the following specification fragment:** *

****

Rover:

2: scan row1

restart if scanFailed

****

"*scanFailed*" becomes true when the scan produces unusable data.

Suppose it becomes true while the "*Agronomist*" is still executing its continuation over sub-sequences 1–2. What happens?

Contrassegna solo un ovale.

- Both agents restart their current execution.
- Only the "Rover" restarts sub-sequence 2.
- The entire pattern terminates with failure.
- The "Rover" proceeds directly to sub-sequence 3.

T2 — Explain a LIrAs pattern

Read the following LIrAs pattern and briefly explain the behavior it describes.

The action and predicate descriptions below define the meaning of the domain-specific terms used in the pattern. You may consult the LIrAs tutorial.

Please spend no more than approximately **4–5 minutes** on this task.

Specification

Available parameters:

- agent type may be "*mobile_robots*" or "*human_waiters*"
- location may be "*kitchen*" or "*table1*";
- resource refers to "*mealTray*".

Actions:

- *moveTo* location: moves the executing agent to the specified location;
- *load* resource: picks up and carries the specified resource;
- *prepare* location: prepares the specified location for service;
- *stop*: keeps the agent still;
- *serve* resource: serves all items from the specified resource. When this action completes, *trayServed*(resource) becomes true.

Predicates:

- *trayServed*(resource) is true when all items from the specified tray have been served.

****

Table_Service

agents:
mobile_robots: Robot
human_waiters: Waiter
locations: kitchen, table1
resources: mealTray

Robot:
1: moveTo kitchen
load mealTray
2: moveTo table1
3: moveTo kitchen if trayServed(mealTray) else stop

Waiter:
1-2: moveTo table1
prepare table1
3: serve mealTray

****

15. Briefly explain, in your own words, the behavior encoded by this LIrAs pattern. *

Focus on how the "*Robot*" and the "*Waiter*" coordinate, including any relevant waiting or termination behavior. Your answer should be approximately **2–4 sentences**. You do not need to explain every line separately.

T3 — Debug (T3A) and Correct (T3B) a LIrAs Pattern

Task, you will inspect a LIrAs pattern that does not correctly represent the intended scenario.

The specification contains **exactly three errors**. First, identify the errors. Then, rewrite the corrected complete pattern.

Use only the actions, predicates, and parameter values provided below.

Please spend no more than approximately **7 minutes** on this task. If you are unsure, submit your best attempt and continue.

Scenario Description

Consider the collaborative placement of a heavy package involving a ground "*Robot*" and a "*Drone*".

The "*Robot*" moves to the "*storageArea*", picks up the "*package*", and transports it to the "*loadingBay*".

At the same time, the "*Drone*" takes off, moves to the "*loadingBay*", and inspects the placement area. The inspection may complete normally, but it must terminate early as soon as the placement area is found clear.

The "*Drone*" must perform these activities independently over the "*Robot*". The two agents must synchronize only before beginning the final placement phase.

During the final phase, the "*Robot*" begins placing the package. The "*Drone*" first moves into a hovering position above the "Robot". If an obstacle is detected, the "*Drone*" must skip the remaining actions in its current sub-sequence, allowing the "*Robot*" to continue alone. Otherwise, the "*Drone*" assists the placement.

If the assistance attempt fails, the "*Drone*" must repeat its own current sub-sequence. The "*Robot*" must not be restarted.

The pattern must terminate with a failure outcome if the system-wide emergency stop becomes active.

- agent type may be *"robots*" or "*drones*";
- location may be "*storageArea"* or "*loadingBay"*;
- resource refers to "*package"*.

Actions

- *moveTo* location: moves the executing agent to the specified location;
- *pick* resource: picks up and carries the specified resource;
- *takeOff*: causes the executing agent to take off;
- *inspect* location: inspects the specified location;
- *hover* agent: moves into a hovering position above the specified agent;

- *place* resource: places the specified resource at the agent's current location;
- *assistPlace* resource: assists another agent in placing the specified resource.

Predicates

- *placementAreaClear*(location) is true when the specified placement area has been found clear;
- *obstacleDetected* is true when the "*Drone*" detects an obstacle that makes assistance unsafe;
- *assistanceSuccessful* is true when the "*Drone's*" assistance attempt succeeds;
- *assistanceFailed* is true when the "*Drone's*" assistance attempt fails;
- *emergencyStopActive* is true when the system-wide emergency stop is active.

****
Collaborative_Package_Placement

agents:
  robots: Robot
  drones: Drone
locations: storageArea, loadingBay
resources: package

Robot:
  1: moveTo storageArea
  2: pick package
  3: moveTo loadingBay
  4: place package

Drone:
  1-2: takeOff
      moveTo loadingBay
      inspect loadingBay until placementAreaClear(loadingBay)
  4: hover Robot
    assistPlace package
    skip if obstacleDetected
    restart if assistanceSuccessful

failure emergencyStopActive

****

16. Which three changes are required for the specification to match the scenario? Select exactly three. *

Seleziona tutte le voci applicabili.

- [ ] The "Robot" should also use a continuation over sub-sequences 1–3.
- [ ] The "Drone" continuation should extend through sub-sequence 3.
- [ ] "place package" and "assistPlace package" should use different labels.
- [ ] "skip if obstacleDetected" should appear before "assistPlace package".
- [ ] "restart" should test "assistanceFailed" instead of "assistanceSuccessful".
- [ ] "inspect loadingBay" should use "restart" instead of "until".

17. **Rewrite the complete LIrAs pattern after correcting the three errors.** *

You may copy the original specification and edit only the incorrect parts. Do not change declarations or instructions that are already consistent with the scenario.

Exact indentation, spacing, and blank lines will not be evaluated.

T4 — Write LIrAs patterns

In this task, you will translate two short scenarios into a complete LIrAs pattern.

You may consult the tutorial. Use only the actions, predicate, and parameter values provided below.

Exact indentation, spacing, and blank lines will not be evaluated. However, action names and parameter names should be used as provided.

Please spend no more than approximately **6 minutes (each)** on this task. If you are unsure, submit your best attempt and continue.

Scenario 1

Consider a guided object-transport task involving a "*Human*" and a mobile "*Robot*".

An object is initially located in the "*bedroom*". The "*Human*" and the "*Robot*" must start moving to the "*bedroom*" at the same time. After both have arrived, the "*Robot*" picks up the "*object*". No action is required from the "*Human*" during the pickup.

Once the "*Robot*" has picked up the object, the "*Human*" moves to the "*kitchen*", and the "*Robot*" follows the "*Human*" there while carrying the object.

The pattern must terminate with a failure outcome if the system-wide emergency stop becomes active.

- agent type may be "*humans"* or "*robots*";

Actions

- *moveTo* location: moves the executing agent to the specified location;
- *pick* resource: picks up and carries the specified resource;
- *follow* agent (location): follows the specified agent to the specified location.

Predicate

- *emergencyStopActive* is true when the system-wide emergency stop is active.

18. Write a complete LIrAs specification named "*Guided_Object_Transport*". *

Your specification should include:
the agent, location, and resource declarations; one sequence for the "*Human*" and one for the "*Robot*"; the failure condition.

Write the complete specification in the field below.

Scenario 2

Consider an electric-vehicle charging task involving an autonomous service "*Robot*" and a human "*Operator*".

At the beginning of the pattern, the "*Robot*" moves to the "*chargingStation*". At the same time, the "*Operator*" moves to the "*storageArea*", collects the "*chargingCable*", and then moves to the "*chargingStation*".

As soon as the "*Robot*" reaches the charging station, it must begin scanning for the charging port without waiting for the "*Operator*". The scan must stop when the port has been located.

The agents must wait for each other only after the "Robot" has located the port and the "*Operator*" has reached the charging station.

The "*Robot*" then opens the charging-port cover. No action is required from the "*Operator*" during this step.

Finally, the "*Robot*" holds the cover open while the "*Operator*" connects the charging cable. These two actions must start at the same time.

- agent type may be "*robots*" or *"humans*";
- location may be "*storageArea"* or "*chargingStation*";
- resource refers to "*chargingCable"*.

Actions:

- *moveTo* location: moves the executing agent to the specified location;
- *collect* resource: collects and carries the specified resource;
- *scanChargingPort*: continuously scans for the vehicle's charging port;
- *openPortCover*: opens the vehicle's charging-port cover;
- *holdPortCover*: holds the charging-port cover open while the action is active;
- *connect* resource: connects the specified charging cable.

Predicates:

- *portLocated* is true when the "*Robot*" has located the charging port.

19. Write a complete LIrAs specification named "*EV_Charging_Assistance*". *

    Your specification must:
    declare the agents, locations, and resource; include one sequence for the "*Robot*" and one for the "*Operator*"; use a continuation to allow the "*Operator*" to proceed without synchronizing while the "*Robot*" moves and scans; use until to stop the port-scanning action.

    Write the complete specification in the field below.

Final feedback

Thank you for completing the study.

The following questions concern your overall experience with LIrAs and with the tutorial. Please consider all the study tasks together.

There are no correct or incorrect answers.

Estimated completion time: approximately **3–4 minutes**.

20. **Overall, how much mental effort did understanding and using LIrAs require?** *
Consider the amount of concentration and reasoning you needed to understand the notation and complete the tasks.

Contrassegna solo un ovale.

- Very low — I could understand and use LIrAs with little concentration or re-reading.
- Low — I needed some attention, but using LIrAs was generally straightforward.
- Moderate — I needed sustained attention and occasionally had to reconsider some rules or examples.
- High — I often had to reason carefully or revisit the provided material to complete the tasks.
- Very high — Understanding and using LIrAs required substantial concentration and repeated reasoning or checking.

21. Please indicate how much you agree or disagree with the following statements about LIrAs. *

Contrassegna solo un ovale per riga.

	Strongly disagree	Disagree	Neither agree nor disagree	Agree	Strongly agree
The overall syntax of LIrAs was easy to understand.	◯	◯	◯	◯	◯
It was clear how to translate a scenario description into a LIrAs specification.	◯	◯	◯	◯	◯
Synchronization points between agents were easy to identify.	◯	◯	◯	◯	◯
Continuations clearly expressed when agents could proceed independently.	◯	◯	◯	◯	◯
The behavior of conditional constructs, such as if/else and until, was predictable.	◯	◯	◯	◯	◯
The behavior of sub-sequence control constructs, such as skip and restart, was predictable.	◯	◯	◯	◯	◯
I felt confident creating and modifying LIrAs patterns.	◯	◯	◯	◯	◯

22. Which aspect of LIrAs was the most difficult to understand or use? *

Contrassegna solo un ovale.

- Basic pattern structure and declarations
- Actions, predicates, and parameters
- Sub-sequences and synchronization
- Continuations
- Conditional constructs (if/else and until)
- Sub-sequence control constructs (skip and restart)
- Success and failure conditions
- No aspect was particularly difficult

23. Please indicate how much you agree or disagree with the following statements about the LIrAs tutorial. *

Contrassegna solo un ovale per riga.

	Strongly disagree	Disagree	Neither agree nor disagree	Agree	Strongly agree
The tutorial was clear and easy to follow.	◯	◯	◯	◯	◯
The explanation of actions, predicates, and parameters was clear.	◯	◯	◯	◯	◯
The examples helped me understand the LIrAs constructs.	◯	◯	◯	◯	◯
The tutorial provided enough information to complete the study tasks.	◯	◯	◯	◯	◯

24. **What changes would most improve LIrAs? (Max. 3)**

You may comment on its syntax, constructs, readability, or ease of use.

Questi contenuti non sono creati né avallati da Google.

Google Moduli